

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE:CSA5115 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I **Nissanki Sri Venkata Lakshmi Kavya (192373036)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

N.S.V.L.Kavya

Annexure A Case Study

1.A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.

Analysis of MD5 Weaknesses and Secure SHA-Based Approach

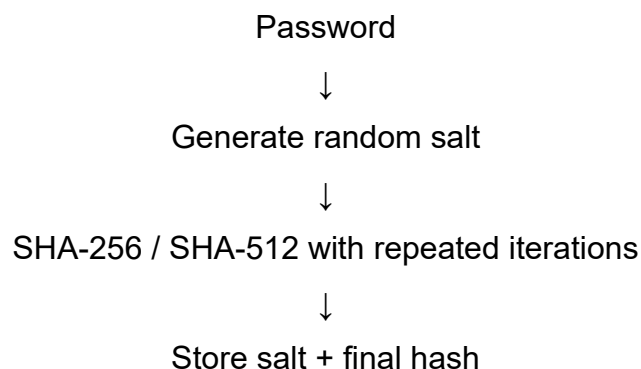
Weaknesses of MD5:

1. **MD5 is cryptographically broken** and has known collision vulnerabilities.
2. It is **very fast**, allowing attackers to try millions or billions of password guesses quickly.
3. MD5 does not provide **salting**, so the same password produces the same hash.
4. Attackers can use **rainbow tables** and precomputed hashes to crack passwords.
5. Once the database is leaked, weak or common passwords can be recovered relatively easily.

Proposed Secure Approach

A better approach is to use a **SHA-based password hashing method with a unique salt**.

For example:



For each user:

hash = SHA-256(password + salt)

For stronger protection, apply **many iterations** of SHA-256 (such as PBKDF2-HMAC-SHA-256), which makes password guessing much slower.

MD5

SHA-based approach

Very fast

Intentionally slower with iterations

Vulnerable to collisions

SHA-256/SHA-512 are much stronger

Usually used without salt

Unique salt for every password

Vulnerable to rainbow tables

Salt makes precomputed attacks much harder

Easy to brute-force

Repeated hashing increases attack cost

Security Improvements

MD5	SHA-based approach
Very fast	Intentionally slower with iterations
Vulnerable to collisions	SHA-256/SHA-512 are much stronger
Usually used without salt	Unique salt for every password
Vulnerable to rainbow tables	Salt makes precomputed attacks much harder
Easy to brute-force	Repeated hashing increases attack cost

Conclusion

MD5 is unsuitable for storing passwords because it is fast and has serious cryptographic weaknesses. The company should migrate to **PBKDF2-HMAC-SHA-256 with a unique random salt for every password**. In practice, dedicated password-hashing algorithms such as Argon2id, scrypt, or bcrypt are generally preferred, but PBKDF2-HMAC-SHA-256 is a strong SHA-based choice when the requirement specifically calls for SHA techniques.

2.A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.

Analysis of Digital Signature Flaws and Improvements

Possible flaws:

1. **Weak or compromised private keys** may allow attackers to create valid signatures.

2. The application may not be **properly verifying the signature** before approving transactions.
3. The **transaction details may not be included in the signed data**, allowing attackers to modify the amount or account number.
4. Poor **key storage and management** can expose private keys.
5. Weak or outdated algorithms may make signatures vulnerable to attacks.
6. Lack of **timestamp, nonce, or transaction ID** can allow replay of previously valid transactions.

Recommended Improvements

1. Use modern standards such as **RSA-PSS with SHA-256/SHA-384** or **ECDSA with SHA-256**.
2. Store private keys securely using **HSMs (Hardware Security Modules)** or secure key-management systems.
3. Sign the complete transaction data, including **sender, receiver, amount, transaction ID, and timestamp**.
4. Implement **nonce/transaction ID and timestamp validation** to prevent replay attacks.
5. Verify the digital signature **before every transaction approval**.
6. Use strong key sizes and regularly **rotate and revoke compromised keys**.

Conclusion

Unauthorized transactions may occur because of weak key management, improper signature verification, replay attacks, or incomplete transaction data being signed. The banking application should use **RSA-PSS or ECDSA with SHA-256**, secure private-key storage, complete transaction signing, and replay protection. These improvements provide stronger **authentication, integrity, and transaction security**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand